

Digital Image Processing

Assignment - 0

August 9, 2026

Name: Harsh Kapoor
Roll No.: 2023112004
Email: harsh.kapoor@research.iiit.ac.in

1 Question 1: Read an Image File into an Array

1.1 Implementation

The first function reads an image file and stores it as a NumPy array.

```
1 def read_image(filename):  
2     img = cv2.imread(filename, cv2.IMREAD_UNCHANGED)  
3     if img is None:  
4         raise FileNotFoundError(  
5             f"Unable to read image: {filename}"  
6         )  
7     return img
```

The function uses OpenCV's `imread()` function. The flag `IMREAD_UNCHANGED` is used so that the image is loaded without unnecessarily modifying its original channel structure.

The image is then loaded using:

```
1 img_array = read_image("input\\input_colored.jpg")  
2 img_shape = img_array.shape  
3 img_type = img_array.dtype
```

The shape of a color image is generally

$$H \times W \times 3,$$

where H represents the height, W represents the width, and the final dimension represents the three color channels.

OpenCV stores color images in BGR channel order.

1.2 Observation

Reading the image into an array allows individual pixels and channels to be accessed and manipulated directly. This array representation forms the basis of the remaining image-processing operations.



Figure 1: Original input image.

2 Question 2: Write an Array into an Image File

2.1 Implementation

The following function writes a NumPy array to an image file:

```
1 def write_image(filename, image_array):
2     cv2.imwrite(filename, image_array)
```

To demonstrate this operation, a random array was generated and added to the original image:

```
1 rand_array = np.random.randint(
2     -255, 255, size=img_shape
3 ).astype(np.int16)
4
5 image_array = np.clip(
6     rand_array + img_array.astype(np.int16),
7     0, 255
8 ).astype(img_type)
9
10 write_image(
11     "output\\output_random.jpg",
12     image_array
13 )
```

The image is temporarily converted to `int16` before performing the arithmetic. This prevents unwanted overflow or underflow that could occur when performing arithmetic directly on an unsigned 8-bit image.

After the modification, the values are clipped to the valid range:

$$0 \leq I(x, y, c) \leq 255.$$

The resulting array is then converted back to the original image data type and written to an image file.

2.2 Observation

This demonstrates the complete process of reading an image, manipulating its numerical representation, and writing the resulting array back to an image file.



Figure 2: Image obtained after modifying the image array and writing it to a file.

3 Question 3: Change the Brightness of an Image

3.1 Implementation

Brightness was modified by adding a constant value to every pixel of the image.

```
1 def change_brightness(image_array, value):
2     new_image = image_array.astype(np.int16) + value
3     new_image = np.clip(new_image, 0, 255)
4     new_image = new_image.astype(img_type)
5     return new_image
```

Mathematically, the transformation is

$$I'(x, y, c) = \text{clip}(I(x, y, c) + \beta, 0, 255),$$

where β is the brightness adjustment.

A positive value of β increases the brightness, while a negative value decreases it.

The function was tested using:

```
1 brighter = change_brightness(img_array, 50)
2 write_image(
```

```

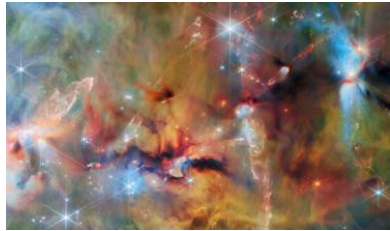
3     "brightness\\output_brighter.jpg",
4     brighter
5 )
6
7 darker = change_brightness(img_array, -50)
8 write_image(
9     "brightness\\output_darker.jpg",
10    darker
11 )

```

3.2 Observation

Increasing the brightness shifts pixel intensities towards higher values, making the image appear lighter. Decreasing the brightness shifts the intensities towards lower values, making the image darker.

Clipping is necessary because pixel values cannot exceed 255 or fall below 0.



(a) Original



(b) Brighter



(c) Darker

Figure 3: Effect of changing image brightness.

4 Question 4: Change the Contrast of an Image

4.1 Implementation

Contrast was changed by multiplying every pixel value by a constant factor.

```

1 def change_contrast(image_array, factor):
2     new_image = factor * image_array
3     new_image = np.clip(new_image, 0, 255)
4     return new_image.astype(img_type)

```

The transformation can be represented as

$$I'(x, y, c) = \text{clip}(\alpha I(x, y, c), 0, 255),$$

where α is the contrast factor.

The function was tested with:

```

1 more_contrast = change_contrast(img_array, 1.25)
2 write_image(
3     "contrast\\output_more_contrast.jpg",
4     more_contrast

```

```

5 )
6
7 less_contrast = change_contrast(img_array, 0.75)
8 write_image(
9     "contrast\\output_less_contrast.jpg",
10    less_contrast
11 )

```

When

$$\alpha > 1,$$

the intensity differences are increased. When

$$0 < \alpha < 1,$$

the intensity range is compressed.

4.2 Observation

Increasing contrast makes the difference between bright and dark regions more pronounced. Decreasing contrast compresses the intensity range and produces a flatter-looking image.



(a) Original



(b) Increased contrast



(c) Decreased contrast

Figure 4: Effect of changing image contrast.

5 Question 5: Convert a Color Image to Grayscale

A color image can be converted to grayscale in several ways. Different methods use different relationships between the three color channels and therefore produce different visual results.

Four methods were implemented:

1. Average method
2. Luminosity method
3. Lightness method
4. Single-channel method

5.1 Average Method

The average method gives equal importance to all three channels:

$$I_{\text{gray}} = \frac{R + G + B}{3}.$$

The implementation is:

```
1 def grayscale_average(img):  
2     return np.mean(img, axis=2).astype(img_type)
```

5.2 Luminosity Method

The luminosity method uses weighted contributions:

$$I_{\text{gray}} = 0.3R + 0.59G + 0.11B.$$

The implementation is:

```
1 def grayscale_luminosity(img):  
2     return (  
3         0.3*img[:, :, 0]  
4         + 0.59*img[:, :, 1]  
5         + 0.11*img[:, :, 2]  
6     ).astype(img_type)
```

The green component receives the highest weight in this formulation.

5.3 Lightness Method

The lightness method uses the average of the maximum and minimum channel values:

$$I_{\text{gray}} = \frac{\max(R, G, B) + \min(R, G, B)}{2}.$$

The implementation is:

```
1 def grayscale_lightness(img):  
2     max_val = np.max(img, axis=2)  
3     min_val = np.min(img, axis=2)  
4  
5     return (  
6         (max_val + min_val) / 2  
7     ).astype(img_type)
```

5.4 Single-Channel Method

Another simple method is to use only one of the color channels:

```
1 def grayscale_single_channel(img, channel=0):  
2     return img[:, :, channel]
```

The implementation was used to obtain three separate outputs:

```

1 red_channel = grayscale_single_channel(
2     img_array, channel=0
3 )
4
5 green_channel = grayscale_single_channel(
6     img_array, channel=1
7 )
8
9 blue_channel = grayscale_single_channel(
10    img_array, channel=2
11 )

```

5.5 Comparison

The different grayscale methods produce different visual appearances.

The average method treats all channels equally. The luminosity method applies different weights to the channels, while the lightness method depends only on the minimum and maximum channel values. The single-channel method completely ignores the other two channels.

Therefore, there is no single universal grayscale conversion. The choice depends on the intended application and the desired visual representation.

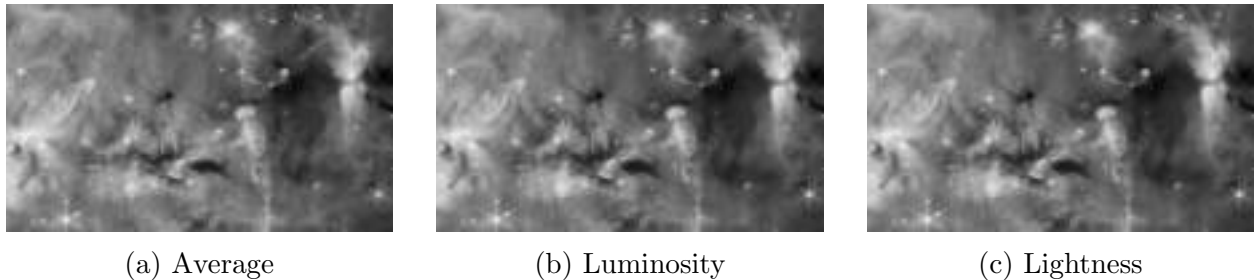


Figure 5: Comparison of grayscale conversion methods.

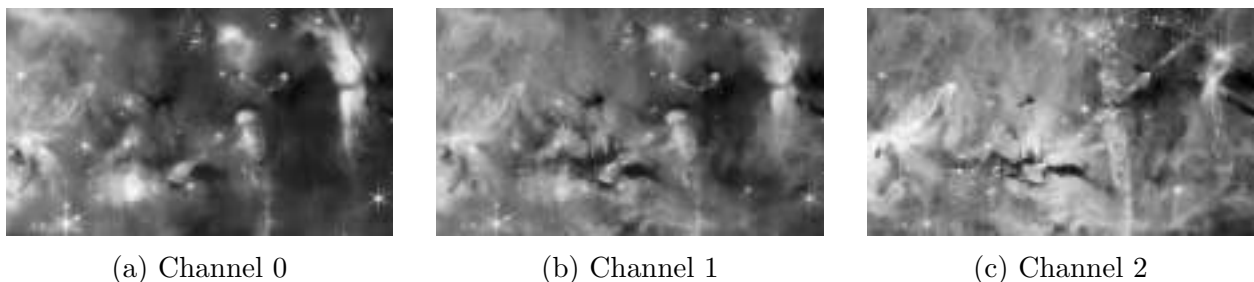


Figure 6: Grayscale images obtained from individual color channels.

6 Question 6: Convert Grayscale to Color using Pseudo-Color Mapping

6.1 Implementation

Pseudo-coloring maps grayscale intensity values to artificial colors. It can make intensity variations easier to distinguish visually.

First, the grayscale image is normalized to the range $[0, 1]$:

$$g = \frac{I_{\text{gray}}}{255}.$$

The implementation is:

```
1 def grayscale_to_pseudo(gray_img):
2     gray_img = gray_img.astype(img_type)
3
4     h, w = gray_img.shape
5     color_img = np.zeros(
6         (h, w, 3),
7         dtype=img_type
8     )
9
10    gray = gray_img.astype(
11        np.float32
12    ) / 255.0
```

Three output channels are then generated using piecewise functions.
For the first channel,

$$C_1 = 255 \max(1.5 - |4g - 1|, 0).$$

Similarly,

$$C_2 = 255 \max(1.5 - |4g - 2|, 0)$$

and

$$C_3 = 255 \max(1.5 - |4g - 3|, 0).$$

The values are clipped to the valid range $[0, 255]$.

```
1 color_img[:, :, 0] = np.clip(
2     255 * np.maximum(
3         1.5 - np.abs(4 * gray - 1),
4         0
5     ),
6     0, 255
7 ).astype(img_type)
```

The same mapping is applied to the other two channels.

The pseudo-color mapping was applied to the average, luminosity, and lightness grayscale images.

6.2 Observation

Pseudo-coloring does not recover the original colors of the image. Instead, it assigns artificial colors according to grayscale intensity. This can make intensity differences more visually prominent.

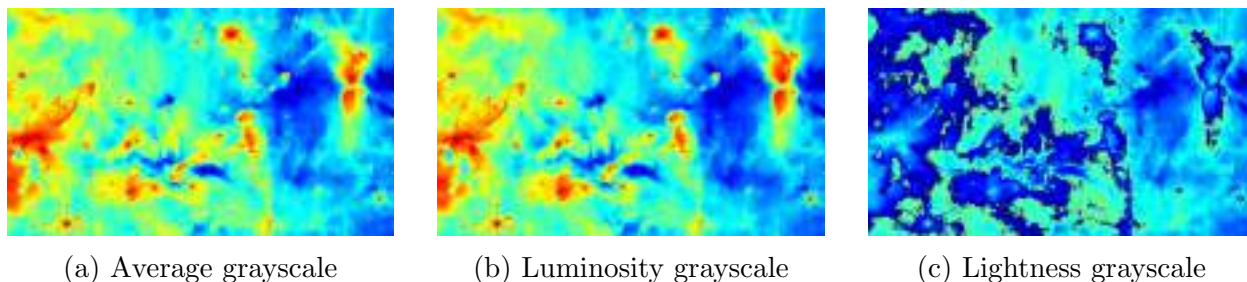


Figure 7: Pseudo-color mapping applied to different grayscale images.

7 Question 7: Green-Screen Background Replacement

7.1 Green Pixel Detection

The first step is to identify pixels belonging to the green-screen background.

The implemented function is:

```
1 def detect_green_pixels(image, green_thresh):
2     R = image[:, :, 0].astype(np.int16)
3     G = image[:, :, 1].astype(np.int16)
4     B = image[:, :, 2].astype(np.int16)
5
6     mask = (
7         G - np.maximum(R, B)
8     ) >= green_thresh
9
10    return mask
```

A pixel is classified as green when

$$G - \max(R, B) \geq T,$$

where T is the green-screen threshold.

A threshold of 200 was used in the experiment.

The resulting mask is a Boolean array indicating which pixels should be replaced.

7.2 Manual Image Resizing using Bilinear Interpolation

Since external image-processing libraries were not used for resizing, the background image was resized manually using bilinear interpolation.

The function first determines the scaling factors:

$$s_y = \frac{H}{H_t}, \quad s_x = \frac{W}{W_t},$$

where (H, W) are the dimensions of the source image and (H_t, W_t) are the dimensions of the target image.

```

1 def resize_to_match(img, target_shape):
2     target_h, target_w = target_shape[:2]
3     h, w = img.shape[:2]
4
5     scale_h = h / target_h
6     scale_w = w / target_w

```

For every pixel in the output image, its corresponding floating-point position in the source image is calculated:

$$y_s = y s_y, \quad x_s = x s_x.$$

The four neighboring source pixels are then identified:

$$(x_0, y_0), \quad (x_1, y_0), \quad (x_0, y_1), \quad (x_1, y_1).$$

The fractional positions are

$$f_x = x_s - x_0, \quad f_y = y_s - y_0.$$

The output pixel is obtained using bilinear interpolation:

$$\begin{aligned}
 I(x, y) = & (1 - f_y)(1 - f_x)I(x_0, y_0) \\
 & + (1 - f_y)f_x I(x_1, y_0) \\
 & + f_y(1 - f_x)I(x_0, y_1) \\
 & + f_y f_x I(x_1, y_1).
 \end{aligned}$$

The implementation performs this calculation explicitly for every output pixel.

```

1 for y in range(target_h):
2     for x in range(target_w):
3
4         src_y = y * scale_h
5         src_x = x * scale_w
6
7         y0 = int(src_y)
8         y1 = min(int(src_y) + 1, h - 1)
9
10        x0 = int(src_x)
11        x1 = min(int(src_x) + 1, w - 1)
12
13        fy = src_y - y0
14        fx = src_x - x0
15
16        resized[y, x] = (
17            (1 - fy) * (1 - fx) * img[y0, x0]
18            + (1 - fy) * fx * img[y0, x1]

```

```

19         + fy * (1 - fx) * img[y1, x0]
20         + fy * fx * img[y1, x1]
21     ).astype(img.dtype)

```

This was an important part of the implementation because the resizing operation was performed without relying on PIL or another image-processing library.

7.3 Green-Screen Replacement

After obtaining the green-screen mask and resizing the background, the corresponding background pixels are substituted:

```

1 def replace_green_screen(
2     foreground,
3     background,
4     green_thresh=200):
5
6     mask = detect_green_pixels(
7         foreground,
8         green_thresh
9     )
10
11     background_resized = resize_to_match(
12         background,
13         foreground.shape
14     )
15
16     result = foreground.copy()
17     result[mask] = background_resized[mask]
18
19     return result

```

Mathematically,

$$I_{\text{output}}(x, y) = \begin{cases} I_{\text{background}}(x, y), & \text{if } M(x, y) = 1, \\ I_{\text{foreground}}(x, y), & \text{if } M(x, y) = 0. \end{cases}$$

The final experiment was performed using:

```

1 foreground_image = read_image(
2     'input\\input_bg.jpg'
3 )
4
5 background_image = read_image(
6     'input\\input_colored.jpg'
7 )
8
9 result_image = replace_green_screen(
10     foreground_image,
11     background_image
12 )
13
14 write_image(
15     'greenscreen\\merged_image.jpg',

```

```
16     result_image
17 )
```

7.4 Observation

The quality of the green-screen replacement depends strongly on the threshold. A high threshold selects only strongly green pixels, whereas a lower threshold would select a larger range of green shades.

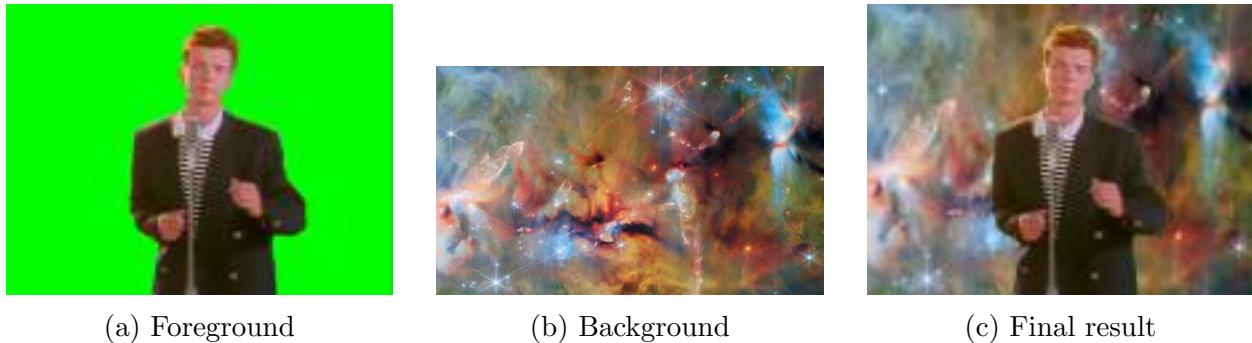


Figure 8: Green-screen background replacement.

8 Question 8: Convert a Video into a Sequence of Images and Write it Back

8.1 Video to Frames

A video can be represented as a sequence of individual image frames. The function `video_to_frames()` extracts these frames.

```
1 def video_to_frames(video_path):
2     cap = cv2.VideoCapture(video_path)
3     frames = []
4
5     while True:
6         ret, frame = cap.read()
7
8         if not ret:
9             break
10
11         frame_rgb = cv2.cvtColor(
12             frame,
13             cv2.COLOR_BGR2RGB
14         )
15
16         frames.append(frame_rgb)
17
18     cap.release()
19     return frames
```

OpenCV reads the frames in BGR format. Therefore, each frame is converted to RGB before being stored.

The extracted frames are stored in a Python list.

8.2 Frames to Video

The reverse operation is performed using:

```
1 def frames_to_video(  
2     frames,  
3     output_path,  
4     fps=30):  
5  
6     if not frames:  
7         raise ValueError(  
8             "No frames to write!"  
9         )  
10  
11     height, width, _ = frames[0].shape  
12  
13     fourcc = cv2.VideoWriter_fourcc(  
14         *"mp4v"  
15     )  
16  
17     out = cv2.VideoWriter(  
18         output_path,  
19         fourcc,  
20         fps,  
21         (width, height)  
22     )  
23  
24     for frame in frames:  
25  
26         frame_bgr = cv2.cvtColor(  
27             frame,  
28             cv2.COLOR_RGB2BGR  
29         )  
30  
31         out.write(frame_bgr)  
32  
33     out.release()
```

The frames are converted back from RGB to BGR before being passed to OpenCV's video writer.

The complete experiment was performed using:

```
1 frames = video_to_frames(  
2     "input\\video.mp4"  
3 )  
4  
5 print(  
6     f"Extracted {len(frames)} frames"  
7 )  
8
```

```
9 frames_to_video(  
10     frames,  
11     "output\\video.mp4",  
12     fps=10  
13 )
```

8.3 Observation

This experiment demonstrates that a video can be represented as an ordered sequence of image arrays:

$$V = \{I_0, I_1, \dots, I_{N-1}\}.$$

Once the video has been converted into frames, each frame can be independently manipulated before reconstructing the video.

The duration of the reconstructed video depends on the number of frames and the selected frame rate:

$$T = \frac{N}{f}.$$



Figure 9: Example frame extracted from the input video.

9 Question 9: Create a One-Second Transition Video

Three different image transitions were implemented:

1. Fade transition
2. Checkerboard transition
3. Slide transition

The transition video uses

$$\text{frames_count} = \text{fps}.$$

Therefore, with the default frame rate of 30 fps, the generated transition contains 30 frames and has a duration of approximately one second.

9.1 Fade Transition

The fade transition uses linear interpolation between the two images.

For images I_1 and I_2 , the output frame is

$$I_i = (1 - \alpha_i)I_1 + \alpha_i I_2,$$

where

$$\alpha_i = \frac{i}{N - 1}.$$

The implementation is:

```
1 def fade_transition(  
2     img1,  
3     img2,  
4     frames_count):  
5  
6     frames = []  
7  
8     img1 = img1.astype(  
9         np.float32  
10    )  
11  
12    img2 = img2.astype(  
13        np.float32  
14    )  
15  
16    for i in range(frames_count):  
17  
18        alpha = i / (  
19            frames_count - 1  
20        )  
21  
22        frame = (
```

```

23         (1 - alpha) * img1
24         + alpha * img2
25     )
26
27     frames.append(
28         frame.astype(np.uint8)
29     )
30
31     return frames

```

At the beginning,

$$\alpha = 0,$$

so the output is the first image. At the end,

$$\alpha = 1,$$

so the output is the second image.

9.2 Checkerboard Transition

The checkerboard transition divides the image into an 8×8 grid. Individual blocks are progressively replaced by the corresponding blocks from the second image.

The transition progress is calculated as

$$p = \frac{i}{N-1}.$$

Each block is assigned an appearance time:

$$t_{x,y} = \frac{x+y}{2S},$$

where S is the number of squares along one dimension.

A block is replaced when

$$p \geq t_{x,y}.$$

The implementation is:

```

1 def checkerboard_transition(
2     img1,
3     img2,
4     frames_count,
5     squares=8):
6
7     frames = []
8
9     height, width, _ = img1.shape
10
11     block_w = width // squares
12     block_h = height // squares
13

```



```

14     for i in range(frames_count):
15
16         progress = i / (
17             frames_count - 1
18         )
19
20         frame = img1.copy()
21
22         for y in range(squares):
23             for x in range(squares):
24
25                 appear_time = (
26                     x + y
27                 ) / (squares * 2)
28
29                 if progress >= appear_time:
30
31                     x_start = x * block_w
32                     y_start = y * block_h
33
34                     frame[
35                         y_start:y_start + block_h,
36                         x_start:x_start + block_w
37                     ] = img2[
38                         y_start:y_start + block_h,
39                         x_start:x_start + block_w
40                     ]
41
42             frames.append(frame)
43
44     return frames

```

This produces a block-based transition in which different regions of the second image appear at different times.

9.3 Slide Transition

In the slide transition, the second image progressively enters from one side while the first image moves out.

The horizontal offset is calculated as

$$d = \lfloor pW \rfloor,$$

where p is the transition progress and W is the image width.

The implementation is:

```

1 def slide_transition(
2     img1,
3     img2,
4     frames_count):
5
6     frames = []
7

```

```

8     height, width, _ = img1.shape
9
10    for i in range(frames_count):
11
12        progress = i / (
13            frames_count - 1
14        )
15
16        offset = int(
17            progress * width
18        )
19
20        frame = np.zeros_like(img1)
21
22        if offset < width:
23
24            frame[:, :width-offset] = \
25                img1[:, offset:]
26
27            frame[:, width-offset:] = \
28                img2[:, :offset]
29
30        frames.append(frame)
31
32    return frames

```

As the transition progresses, an increasingly larger portion of the second image occupies the frame.

9.4 Generating the Transition Videos

The three transitions are selected inside the following function:

```

1 def create_transition_video(
2     img1,
3     img2,
4     transition,
5     output_path,
6     fps=30):
7
8     frames_count = fps
9
10    if transition == "fade":
11
12        frames = fade_transition(
13            img1,
14            img2,
15            frames_count
16        )
17
18    elif transition == "checkerboard":
19
20        frames = checkerboard_transition(
21            img1,

```

```

22         img2,
23         frames_count
24     )
25
26     elif transition == "slide":
27
28         frames = slide_transition(
29             img1,
30             img2,
31             frames_count
32         )
33
34     else:
35         raise ValueError(
36             "Unknown transition"
37         )
38
39     frames = [
40         cv2.cvtColor(
41             frame,
42             cv2.COLOR_BGR2RGB
43         )
44         for frame in frames
45     ]
46
47     frames_to_video(
48         frames,
49         output_path,
50         fps
51     )

```

Before creating the transitions, the second image is resized to match the dimensions of the first image using the manually implemented bilinear interpolation function:

```

1 image1 = read_image(
2     "input\\Image_1.jpg"
3 )
4
5 image2 = read_image(
6     "input\\Image_2.jpg"
7 )
8
9 image2 = resize_to_match(
10     image2,
11     image1.shape
12 )

```

The three transition videos are then generated:

```

1 create_transition_video(
2     image1,
3     image2,
4     "fade",
5     "output\\fade.mp4"
6 )
7

```

```

8 create_transition_video(
9     image1,
10    image2,
11    "checkerboard",
12    "output\\checkerboard.mp4"
13 )
14
15 create_transition_video(
16     image1,
17     image2,
18     "slide",
19     "output\\slide.mp4"
20 )

```

9.5 Comparison of Transitions

The three transitions have different visual characteristics.

- **Fade:** The first image gradually blends into the second image using linear interpolation.
- **Checkerboard:** The second image appears progressively in blocks, creating a discrete spatial transition.
- **Slide:** The second image enters horizontally while the first image moves out of the frame.



(a) Fade



(b) Checkerboard



(c) Slide

Figure 10: Comparison of the implemented image transition techniques.

10 Interesting Learnings

Several useful observations were made during the implementation.

10.1 Images as Arrays

Digital images can be represented directly as numerical arrays. This makes operations such as brightness modification, contrast modification, grayscale conversion, and color mapping possible through mathematical operations on arrays.

10.2 Data Type and Overflow

Image arrays are generally stored using 8-bit unsigned integers. Arithmetic operations can therefore cause overflow or underflow if performed directly. Converting the image to a larger integer type before arithmetic and clipping the result afterwards avoids this problem.

10.3 Bilinear Interpolation

Implementing image resizing manually provided an understanding of how interpolation works at the pixel level. Instead of simply selecting the nearest source pixel, bilinear interpolation considers the four neighboring source pixels and computes a weighted average based on the fractional position.

10.4 Video as a Sequence of Images

A video can be treated as an ordered sequence of frames. This makes it possible to apply image-processing techniques to individual video frames and then reconstruct the processed sequence into a new video.

10.5 Color Channel Ordering

OpenCV represents color images in BGR order. Therefore, explicit conversion between BGR and RGB is required when transferring frames between different stages of the video-processing pipeline.